

Pseudocódigo do calculo de vetores

Esse problema foi retirado do atual conteúdo de Geometria Analítica.

INÍCIO

// Começamos definindo os vetores com os valores que queremos usar

DEFINIR VETORES v1, v2, v3

// Agora, calculamos e mostramos os resultados das seguintes operações:

CALCULAR:

Soma de Vetores // Juntar dois vetores (ex: $v1 + v2$)

Subtração de Vetores // Tirar um vetor do outro (ex: $v2 - v1$)

Multiplicação por Escalar // Multiplicar um vetor por um número simples

Produto Escalar // Multiplicar dois vetores, resultando em um número

Produto Vetorial // Multiplicar dois vetores, resultando em um novo vetor (só funciona com 3D)

Magnitude de Vetores // O 'tamanho' ou 'comprimento' de um vetor

Vetor Unitário // Um vetor que aponta na mesma direção, mas tem tamanho 1

Ângulo entre Vetores // Descobrir a abertura entre dois vetores

FIM

primeira versão do código

```
import numpy as np

# Criando vetores
v1 = np.array([1, 2, 3])
v2 = np.array([4, 5, 6])
v3 = np.array([-1, 0, 2])

print("Vetor 1 (v1):", v1)
print("Vetor 2 (v2):", v2)
print("Vetor 3 (v3):", v3)

# --- Operações Básicas ---

# 1. Adição de vetores
soma_vetores = v1 + v2
print("\nSoma de v1 + v2:", soma_vetores)

# 2. Subtração de vetores
subtracao_vetores = v2 - v1
print("Subtração de v2 - v1:", subtracao_vetores)

# 3. Multiplicação por um escalar
```

```

escalar = 2
multiplicacao_escalar = v1 * escalar
print("Multiplicação de v1 por 2:", multiplicacao_escalar)

# 4. Produto Escalar (Dot Product)
produto_escalar = np.dot(v1, v2)
print("Produto Escalar (v1 . v2):", produto_escalar)

# 5. Produto Vetorial (Cross Product) - Apenas para vetores 3D
produto_vetorial = np.cross(v1, v2)
print("Produto Vetorial (v1 x v2):", produto_vetorial)

# 6. Magnitude (Norma) de um vetor
magnitude_v1 = np.linalg.norm(v1)
print("Magnitude de v1:", magnitude_v1)

magnitude_v3 = np.linalg.norm(v3)
print("Magnitude de v3:", magnitude_v3)

# 7. Vetor Unitário
# Um vetor unitário tem magnitude 1 e a mesma direção do vetor original.
vetor_unitario_v1 = v1 / np.linalg.norm(v1)
print("Vetor unitário de v1:", vetor_unitario_v1)
print("Magnitude do vetor unitário de v1:",
      np.linalg.norm(vetor_unitario_v1))

# 8. Ângulo entre dois vetores
#  $\cos(\theta) = (v1 \cdot v2) / (||v1|| * ||v2||)$ 
cos_theta = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
ang_radianos = np.arccos(cos_theta)
ang_graus = np.degrees(ang_radianos)
print("\nCosseno do ângulo entre v1 e v2:", cos_theta)
print("Ângulo entre v1 e v2 (radianos):", ang_radianos)
print("Ângulo entre v1 e v2 (graus):", ang_graus)

```

código com gráfico

```

import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import pandas as pd
import seaborn as sns

```

```

# Helper function para plotagem de vetores 3D
def plot_vectors_3d(vectors_data, title="Visualização de Vetores 3D",
                    x_label="X", y_label="Y", z_label="Z"):
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    max_coord = 0
    for vec, color, label in vectors_data:
        ax.quiver(0, 0, 0, vec[0], vec[1], vec[2], color=color,
                  label=label, arrow_length_ratio=0.08, linewidth=2)
        max_coord = max(max_coord, np.max(np.abs(vec))) if len(vec) > 0
    else max_coord

    # Ajusta os limites para que todos os vetores sejam visíveis
    limit = max_coord * 1.2 if max_coord > 0 else 1
    ax.set_xlim([-limit, limit])
    ax.set_ylim([-limit, limit])
    ax.set_zlim([-limit, limit])
    ax.set_xlabel(x_label)
    ax.set_ylabel(y_label)
    ax.set_zlabel(z_label)
    ax.set_title(title)
    ax.legend()
    ax.view_init(elev=20, azim=30) # Um bom ângulo de visão padrão
    plt.grid(True)
    plt.show()

# ### Visualização das Operações

# Criando dados para o gráfico de barras (magnitudes)
magnitudes = [
    {"vetor": "v1", "magnitude": np.linalg.norm(v1)},
    {"vetor": "v2", "magnitude": np.linalg.norm(v2)},
    {"vetor": "v3", "magnitude": np.linalg.norm(v3)}
]

df_magnitudes = pd.DataFrame(magnitudes)

plt.figure(figsize=(8, 5))
sns.barplot(x='vetor', y='magnitude', data=df_magnitudes,
            palette='viridis')
plt.title('Magnitude dos Vetores v1, v2 e v3')
plt.xlabel('Vetor')
plt.ylabel('Magnitude')

```

```

plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# ##### 1. Adição de Vetores ( $v_1 + v_2$ )

# Visualização da Adição de Vetores
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (soma_vetores, 'b', 'Soma ( $v_1 + v_2$ )')
]
plot_vectors_3d(vectors_to_plot, title='Adição de Vetores:  $v_1 + v_2$ ')

# ##### 2. Subtração de Vetores ( $v_2 - v_1$ )

# Visualização da Subtração de Vetores
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (subtracao_vetores, 'b', 'Subtração ( $v_2 - v_1$ )')
]
plot_vectors_3d(vectors_to_plot, title='Subtração de Vetores:  $v_2 - v_1$ ')

# ##### 3. Multiplicação por um Escalar ( $v_1 * 2$ )

# Visualização da Multiplicação por um Escalar
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (multiplicacao_escalar, 'g', f'V1 * {escalar}')
]
plot_vectors_3d(vectors_to_plot, title=f'Multiplicação por Escalar:  $v_1 * \{escalar\}$ ')

# ##### 4. Produto Escalar ( $v_1 \cdot v_2$ )

# Visualização do Produto Escalar (mostrar vetores e ângulo)
fig = plt.figure(figsize=(10, 8))
ax = fig.add_subplot(111, projection='3d')

vectors_to_plot_2d = [
    (v1, 'r', 'Vetor v1'),

```

```

        (v2, 'g', 'Vetor v2')
    ]
    max_coord_2d = 0
    for vec, color, label in vectors_to_plot_2d:
        ax.quiver(0, 0, 0, vec[0], vec[1], vec[2], color=color,
                  label=label, arrow_length_ratio=0.08, linewidth=2)
        max_coord_2d = max(max_coord_2d, np.max(np.abs(vec))) if len(vec) > 0
    else max_coord_2d

    limit_2d = max_coord_2d * 1.2 if max_coord_2d > 0 else 1
    ax.set_xlim([-limit_2d, limit_2d])
    ax.set_ylim([-limit_2d, limit_2d])
    ax.set_zlim([-limit_2d, limit_2d])
    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_title(f'Produto Escalar:  $v_1 \cdot v_2 = \{\text{produto\_escalar:.2f}\}$ \nÂngulo:
    {\ang_graus:.2f}°')
    ax.legend()
    ax.view_init(elev=20, azim=30)
    plt.grid(True)
    plt.show()

```

```

# ##### 5. Produto Vetorial ( $v_1 \times v_2$ )

```

```

# Visualização do Produto Vetorial
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (produto_vetorial, 'b', 'Produto Vetorial ( $v_1 \times v_2$ )')
]
plot_vectors_3d(vectors_to_plot, title='Produto Vetorial:  $v_1 \times v_2$ ')

```

```

# ##### 6. Vetor Unitário de  $v_1$ 

```

```

# Visualização do Vetor Unitário
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (vetor_unitario_v1, 'g', 'Vetor Unitário de  $v_1$ ')
]
plot_vectors_3d(vectors_to_plot, title='Vetor Unitário de  $v_1$ ')

```

```

código otimizado
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import pandas as pd
import seaborn as sns

# Definição dos vetores
v1 = np.array([3, 2, 5])
v2 = np.array([1, 4, 2])
v3 = np.array([-2, 6, 3])

# Operações de vetores
soma_vetores = v1 + v2
subtracao_vetores = v2 - v1

escalar = 2
multiplicacao_escalar = v1 * escalar

produto_escalar = np.dot(v1, v2)

# Cálculo do ângulo entre v1 e v2
cos_theta = np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))
ang_rad = np.arccos(np.clip(cos_theta, -1.0, 1.0)) # Limitar para evitar
erros de ponto flutuante
ang_graus = np.degrees(ang_rad)

produto_vetorial = np.cross(v1, v2)

vetor_unitario_v1 = v1 / np.linalg.norm(v1)

# Helper function para plotagem de vetores 3D
def plot_vectors_3d(vectors_data, title="Visualização de Vetores 3D",
                    x_label="X", y_label="Y", z_label="Z"):
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    max_coord = 0
    for vec, color, label in vectors_data:
        ax.quiver(0, 0, 0, vec[0], vec[1], vec[2], color=color,
                  label=label, arrow_length_ratio=0.08, linewidth=2)
        max_coord = max(max_coord, np.max(np.abs(vec))) if len(vec) > 0
    else max_coord

```

```

# Ajusta os limites para que todos os vetores sejam visíveis
limit = max_coord * 1.2 if max_coord > 0 else 1
ax.set_xlim([-limit, limit])
ax.set_ylim([-limit, limit])
ax.set_zlim([-limit, limit])
ax.set_xlabel(x_label)
ax.set_ylabel(y_label)
ax.set_zlabel(z_label)
ax.set_title(title)
ax.legend()
ax.view_init(elev=20, azim=30) # Um bom ângulo de visão padrão
plt.grid(True)
plt.show()

# ### Visualização das Operações

# Criando dados para o gráfico de barras (magnitudes)
magnitudes = [
    {"vetor": "v1", "magnitude": np.linalg.norm(v1)},
    {"vetor": "v2", "magnitude": np.linalg.norm(v2)},
    {"vetor": "v3", "magnitude": np.linalg.norm(v3)}
]

df_magnitudes = pd.DataFrame(magnitudes)

plt.figure(figsize=(8, 5))
sns.barplot(x='vetor', y='magnitude', data=df_magnitudes,
palette='viridis', hue='vetor', legend=False)
plt.title('Magnitude dos Vetores v1, v2 e v3')
plt.xlabel('Vetor')
plt.ylabel('Magnitude')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()

# ##### 1. Adição de Vetores (v1 + v2)

# Visualização da Adição de Vetores
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (soma_vetores, 'b', 'Soma (v1 + v2)')
]

plot_vectors_3d(vectors_to_plot, title='Adição de Vetores: v1 + v2')

```

```

# ##### 2. Subtração de Vetores ( $v_2 - v_1$ )

# Visualização da Subtração de Vetores
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (subtracao_vetores, 'b', 'Subtração ( $v_2 - v_1$ )')
]
plot_vectors_3d(vectors_to_plot, title='Subtração de Vetores:  $v_2 - v_1$ ')

# ##### 3. Multiplicação por um Escalar ( $v_1 * 2$ )

# Visualização da Multiplicação por um Escalar
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (multiplicacao_escalar, 'g', f'V1 * {escalar}')
]
plot_vectors_3d(vectors_to_plot, title=f'Multiplicação por Escalar:  $v_1 * \{escalar\}$ ')

# ##### 4. Produto Escalar ( $v_1 \cdot v_2$ )

# Visualização do Produto Escalar (mostrar vetores e ângulo)
vectors_to_plot_2d = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2')
]
plot_vectors_3d(vectors_to_plot_2d, title=f'Produto Escalar:  $v_1 \cdot v_2 = \{produto\_escalar:.2f\}$ \nÂngulo:  $\{ang\_graus:.2f\}^\circ$ ')

# ##### 5. Produto Vetorial ( $v_1 \times v_2$ )

# Visualização do Produto Vetorial
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (v2, 'g', 'Vetor v2'),
    (produto_vetorial, 'b', 'Produto Vetorial ( $v_1 \times v_2$ )')
]
plot_vectors_3d(vectors_to_plot, title='Produto Vetorial:  $v_1 \times v_2$ ')

```



```
# #### 6. Vetor Unitário de v1

# Visualização do Vetor Unitário
vectors_to_plot = [
    (v1, 'r', 'Vetor v1'),
    (vetor_unitario_v1, 'g', 'Vetor Unitário de v1')
]
plot_vectors_3d(vectors_to_plot, title='Vetor Unitário de v1')
```